

Rapport final RLMinimax

Projet Pac-Man - Algorithmique 4 - 2025/2026

Poids appris, resultats finaux, justification algorithmique et controle de conformite au sujet.

Element	Conclusion
Statut technique	Conforme au sujet : Alpha-Beta + fonction affine + poids appris + comparaison statistique.
Verification	Controle effectue le 14/05/2026 : syntaxe, formule affine, poids, resultats et PDF verifies.
Attention	Le depot Git/Gitesi doit etre confirme separement dans le vrai depot de remise.

Resume tres court

Le projet construit un agent RLMinimax. Il explore les coups avec Alpha-Beta, puis il evalue les etats avec une fonction affine dont les poids ont ete appris par renforcement.

Les resultats relances montrent que RLMinimax obtient un meilleur score moyen que Minimax et AlphaBeta sur tous les layouts testes.

1. Conformite au sujet

Le sujet demande une fonction affine, des facteurs lies a Pac-Man, certains facteurs avec recherche, un agent Minimax avec elagage Alpha-Beta, des poids appris par renforcement et une comparaison statistique.

Exigence	Implementation	Statut
Facteurs xi(s) lies a Pac-Man	7 features : nourriture, fantomes, capsules, score, nourriture restante.	OK
Recherche dans les facteurs	BFS pour distances reelles vers nourriture, capsules et fantomes.	OK
Fonction affine	<code>rl_evaluation_function = sum(ai * xi) + C.</code>	OK
Poids independants de l etat	<code>weights.json</code> contient les poids fixes utilises par RLMinimax.	OK
Minimax Alpha-Beta	RLMinimaxAgent alterne Pac-Man MAX et fantomes MIN avec coupure $\alpha \geq \beta$.	OK
Poids appris par renforcement	<code>train.py</code> simule des parties et ajuste les poids par mise a jour TD.	OK
Pas une politique directe	Le RL apprend les poids ; l action vient ensuite d Alpha-Beta.	OK
Comparaison statistique	<code>compare.py</code> mesure score moyen, winrate, temps, coups et Stop moyen.	OK

Choix important : Stop est ignore quand Pac-Man peut faire un vrai mouvement. Cela evite un comportement immobile et garde l agent actif.

2. Fonction affine et poids actuels

Modele utilise : $f(s) = a_1 \cdot x_1(s) + a_2 \cdot x_2(s) + \dots + a_7 \cdot x_7(s) + C$.

Facteur	Fonction	Poids	Information	Sens
x1	nearest_food_bfs	0.6891	Nourriture proche	Pac-Man cherche la nourriture.
x2	ghosts_within_3	-1.4093	Fantomes dangereux proches	Pac-Man evite le danger.
x3	scared_ghosts_nearby	1.4061	Fantomes effrayes proches	Pac-Man profite des opportunités.
x4	remaining_food	0.4036	Nourriture restante	Finir la carte est encourage.
x5	nearest_capsule_bfs	0.0252	Capsule proche	Effet positif mais faible.
x6	current_score	1.8572	Score courant normalise	Le score officiel reste important.
x7	nearest_dangerous_ghost_bfs	-1.2624	Danger principal proche	Pac-Man reste loin du danger.
C	bias	-0.0834	Correction globale	Constante independante de l etat.

Verification : il y a 7 poids pour 7 features, et la valeur calculee par le code correspond exactement a $\sum(a_i \cdot x_i) + C$.

3. Resultats finaux relances

Les tests ont ete relances le 14/05/2026 avec les poids actuels, sans reentrainement. La graine seed=0 permet une comparaison controlee.

Layout	Depth	N	Minimax	AlphaBeta	RLMinimax	Diff RL
testClassic	1	200	548.5	548.5	552.5	+4.0
smallClassic	2	100	-149.8	-149.8	196.6	+346.4
capsuleClassic	2	100	-228.9	-228.9	-135.2	+93.7
mediumClassic	1	50	14.5	14.5	515.5	+501.0
originalClassic	1	50	348.6	348.6	664.4	+315.7
trickyClassic	1	50	101.3	101.3	293.0	+191.7
contestClassic	1	50	11.3	11.3	244.4	+233.1
minimaxClassic	3	100	-243.5	-243.5	-153.1	+90.4
trappedClassic	1	1000	-392.3	-392.3	-391.4	+0.9
openClassic	1	10	-419.5	-745.4	1057.0	+1476.5

Lecture : la colonne Diff RL est positive partout. RLMinimax fait donc mieux que Minimax en score moyen sur tous les layouts testes.

4. Details des resultats RLMinimax

Layout	RL vs AlphaBeta	Win RL	Temps RL	Stop RL
testClassic	+4.0	100%	0.016s	0.0
smallClassic	+346.4	34%	0.284s	0.0
capsuleClassic	+93.7	12%	0.304s	0.0
mediumClassic	+501.0	28%	0.219s	0.0
originalClassic	+315.7	2%	1.430s	0.0
trickyClassic	+191.7	2%	0.643s	0.0
contestClassic	+233.1	28%	0.214s	0.0
minimaxClassic	+90.4	34%	0.036s	0.0
trappedClassic	+0.9	11%	0.001s	0.0
openClassic	+1802.4	90%	0.521s	0.0

Le winrate n est pas parfait sur toutes les cartes, mais le score moyen est meilleur. C est defendable : les fantomes restent aleatoires et certains layouts sont difficiles.

Sur openClassic, les baselines depassent le timeout choisi. C est une limite a expliquer clairement a l oral.

5. Justification algorithmique

Question	Reponse simple
Pourquoi BFS ?	BFS donne une distance reelle dans le labyrinthe, donc il respecte mieux les murs que Manhattan.
Pourquoi Alpha-Beta ?	Alpha-Beta garde la logique Minimax mais evite des calculs inutiles avec alpha et beta.
Pourquoi une fonction affine ?	Le sujet impose une somme ponderee des facteurs, sans termes croises.
Pourquoi apprendre les poids ?	Les poids doivent venir d un apprentissage, pas d un choix manuel arbitraire.
Pourquoi comparer ?	La comparaison prouve si les poids appris ameliorent le comportement.

A dire a l oral

Notre agent utilise Alpha-Beta pour explorer les coups possibles. Quand il doit evaluer une position, il utilise une fonction affine dont les facteurs viennent du jeu et dont les poids ont ete appris par renforcement.

6. Commandes reproductibles et conclusion

But	Commande
Verifier la syntaxe	<code>python -m py_compile compare.py game.py pacman.py layout.py train.py rlMinimaxAgents.py features.py multiAgents.py ghostAgents.py textDisplay.py util.py</code>
Comparer les agents	<code>python compare.py --num-games 100 --layout smallClassic --depth 2 --baseline both</code>
Lancer RLMinimax	<code>python pacman.py -p RLMinimaxAgent -l testClassic -a depth=1 -q -n 1 -f</code>
Voir les poids	<code>weights.json</code>

Conclusion finale
Le correctif est termine pour la partie technique du projet. Le projet respecte le sujet : fonction affine, facteurs Pac-Man, BFS, Alpha-Beta, poids appris par renforcement, resultats statistiques et justification claire. Le seul point a confirmer hors code est le depot Git/Gitesl.